MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on Laboratory Work no. 7
Operating Systems

Theme: Three Small Programs: Processes, Redirection, a Pipe

Done by: st. gr. FAF-242, **Moroșanu Ion**
Checked by: **Reițman Patricia**, *university assistant*
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
1 FORK, EXEC AND WAIT **4**
1.1 The program run.c 4
1.2 Another program and a missing program 5
1.3 Answers to the questions 6
2 A ZOMBIE ON PURPOSE **7**
2.1 The program zombie.c 7
2.2 The zombie in the process table 7
2.3 Answers to the questions 8
3 REDIRECTION: LS -L > OUT.TXT **9**
3.1 The program redirect.c 9
3.2 The result 9
3.3 Answers to the questions 10
4 A PIPE BETWEEN LS AND WC **11**
4.1 The program pipeline.c 11
4.2 Breaking the pipe on purpose 11
4.3 Optional: three stages 12
4.4 Answers to the questions 12
CONCLUSIONS **14**
REFERENCES **15**

INTRODUCTION

Purpose of the work

The lecture asked three questions: how a process is born, where printf writes, and how the shell runs `ls | wc -l`. The purpose of this laboratory work is to answer them by experiment, with three small C programs that do by hand what the shell does every time a command is typed: create a process, redirect its output and connect two processes with a pipe [1].

The following tasks were set for the work:

a) to run another program in a child process with fork, exec and wait, and to see what happens when exec fails;

b) to create a zombie process on purpose and to watch it in the process table;

c) to redirect the standard output of a program into a file, the way the shell does for `>`;

d) to build the pipeline `ls /etc | wc -l` with one pipe and two child processes, and to break it on purpose.

The work was done in the folder ~/os-lab7 of the Ubuntu 26.04 virtual machine used in the previous laboratory works, as the user wancea. The programs were compiled with gcc 15.2.0 and the options `-Wall -Wextra`, which produced no warnings. The final versions of run.c, zombie.c, redirect.c, pipeline.c and the optional pipeline3.c are in the folder lab7/ of the repository, together with report.md, which holds the answers to the questions in short form.

1 FORK, EXEC AND WAIT

1.1 The program run.c

The program run.c (Figure 1.1) repeats the three steps of the shell for the command `ls -l`. First the parent prints its PID and calls `fork`, which creates a copy of the process. In the child, fork returns 0; the child prints its own PID and the PID of its parent and calls `execlp("ls", "ls", "-l", NULL)`, which replaces the program of the process with ls. In the parent, fork returns the PID of the child, and the parent waits for it with `waitpid` and prints its exit status, extracted with the macro WEXITSTATUS [2], [3].

Both processes call `fflush(stdout)` after printf. When the output does not go to a terminal, printf keeps the text in a buffer inside the process; without the flush, fork would copy that buffer into the child and the same line could be printed twice, and exec would throw away whatever the child had not written yet.

```
wancea@OS-Lab1-2:~/os-lab7$ cat run.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Figure 1.1 – The file run.c

The program was compiled without warnings and run (Figure 1.2). The parent had the PID 4305 and the child the PID 4306, and the child reported 4305 as its parent. Then the listing of ls appeared, written by the same process 4306, which had become ls, and finally the parent reported that child 4306 ended with the status 0. The listing also contains the executable run itself, 16392 bytes, created a moment earlier by gcc.

```
wancea@OS-Lab1-2:~/os-lab7$ gcc --version | head -1
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
wancea@OS-Lab1-2:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4305
child:  my PID is 4306, my parent is 4305
total 40
-rw-rw-r-- 1 wancea wancea  1013 Oct 25 16:59 pipeline.c
-rw-rw-r-- 1 wancea wancea  1363 Oct 25 16:59 pipeline3.c
-rw-rw-r-- 1 wancea wancea   703 Oct 25 16:59 redirect.c
-rwxrwxr-x 1 wancea wancea 16392 Oct 25 17:00 run
-rw-rw-r-- 1 wancea wancea   911 Oct 25 16:59 run.c
-rw-rw-r-- 1 wancea wancea   979 Oct 25 16:59 zombie.c
parent: child 4306 ended with status 0
```

Figure 1.2 – run.c compiled and running ls -l in the child

1.2 Another program and a missing program

The line of execlp was then changed with sed, and `grep -n execlp run.c` confirmed the change before each new build (Figures 1.3 and 1.4). With `execlp("date", "date", NULL)` the child became date: it printed the time of the virtual machine, Sun Oct 25 05:00:58 PM UTC 2026, and the parent again saw the status 0.

```
wancea@OS-Lab1-2:~/os-lab7$ sed -i 's/execlp(.*NULL);/execlp("date", "date", (char
*)NULL);/' run.c
wancea@OS-Lab1-2:~/os-lab7$ grep -n execlp run.c && gcc -Wall -Wextra -o run run.c
&& ./run
18:        execlp("date", "date", (char *)NULL);  /* step 2: become ls */
parent: my PID is 4324
child:  my PID is 4325, my parent is 4324
Sun Oct 25 05:00:58 PM UTC 2026
parent: child 4325 ended with status 0
```

Figure 1.3 – The child runs date instead of ls

With the name nosuchprog exec failed, because no program with this name exists in any folder of PATH. Only in this case did the child continue after the call: perror printed `exec: No such file or directory`, the text of the error ENOENT, and the child ended with `exit(127)`, so the parent reported the status 127 (Figure 1.4). This is the same code that bash uses for a command that is not found, as seen in the third laboratory work. After this test the line was changed back to ls, so the file run.c in the repository is the original version.

```
wancea@OS-Lab1-2:~/os-lab7$ sed -i 's/execlp(.*NULL);/execlp("nosuchprog", "nosuchp
rog", (char *)NULL);/' run.c
wancea@OS-Lab1-2:~/os-lab7$ grep -n execlp run.c && gcc -Wall -Wextra -o run run.c
&& ./run
18:        execlp("nosuchprog", "nosuchprog", (char *)NULL);  /* step 2: become ls
*/
parent: my PID is 4337
child:  my PID is 4338, my parent is 4337
exec: No such file or directory
parent: child 4338 ended with status 127
```

Figure 1.4 – exec of a program that does not exist

1.3 Answers to the questions

1. `perror("exec")` is printed only with nosuchprog, because a successful exec never returns. It replaces the whole program of the child (code, data and stack) with ls or date, so the lines after execlp no longer exist in that process. execlp returns, with the value -1, only when it fails, and only then do perror and exit(127) run.

2. The child prints the PID of the parent process, the program run itself: 4305 in Figure 1.2, which is the number that the parent printed in its first line. The child gets it with `getppid()`.

3. With ls (and with date) the parent sees the status 0, the exit code of a program that succeeded. With nosuchprog it sees 127, the value that the child passed to exit after the failed exec.

2 A ZOMBIE ON PURPOSE

2.1 The program zombie.c

The file zombie.c is a copy of the original run.c with one more line, `sleep(20);`, just before waitpid. `diff run.c zombie.c` shows that this line, number 24, is the only difference (Figure 2.1).

```
wancea@OS-Lab1-2:~/os-lab7$ sed -i 's/execlp(.*NULL);/execlp("ls", "ls", "-l", (cha
r *)NULL);/' run.c
wancea@OS-Lab1-2:~/os-lab7$ grep -n execlp run.c && diff run.c zombie.c
18:        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
23a24
>     sleep(20);                                   /* the new line */
wancea@OS-Lab1-2:~/os-lab7$ gcc -Wall -Wextra -o run run.c && gcc -Wall -Wextra -o
zombie zombie.c && echo compiled
compiled
```

Figure 2.1 – The only difference between run.c and zombie.c

2.2 The zombie in the process table

The program was started in the background with its output sent to /dev/null, so the listing of ls was discarded. Two seconds later `ps -o pid,ppid,stat,cmd` showed the parent ./zombie with the PID 4364 in the state S, sleeping in its sleep(20), and its child 4366 as `[ls] <defunct>` in the state Z (Figure 2.2). The parent of ./zombie is 4034, the bash of the terminal; in the sample output of the handout it is 1, because there the program was started from a shell running as PID 1 in a container.

```
wancea@OS-Lab1-2:~/os-lab7$ ./zombie > /dev/null &
[1] 4364
wancea@OS-Lab1-2:~/os-lab7$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   4034    4002 Ss   /usr/bin/bash
   4364    4034 S    ./zombie
   4366    4364 Z    [ls] <defunct>
   4369    4034 R+   ps -o pid,ppid,stat,cmd
```

Figure 2.2 – The child of zombie in the state Z while the parent sleeps

ls had finished its work long before, but it stayed in the table as a zombie: its memory had been freed, and only its entry with the PID and the exit status was kept for the parent. A second ps a few seconds later still showed it. After the 20 seconds had passed, the third ps no longer listed either ./zombie or [ls], and bash reported `[1]+ Done ./zombie > /dev/null` (Figure 2.3).



Figure 2.3 – The zombie is still there a few seconds later and gone after the 20 seconds

2.3 Answers to the questions

1. In the STAT column ls has the letter Z, which means zombie, or defunct: the process has ended, but its parent has not yet collected its exit status, so the kernel keeps its entry in the process table.

2. No. After 20 seconds the line `[ls] <defunct>` was gone (Figure 2.3). It was removed by the line `waitpid(pid, &status, 0);` of zombie.c: waitpid reads the exit status of the child, and only then does the kernel free its last entry. Right after that the parent also ended.

3 REDIRECTION: LS -L > OUT.TXT

3.1 The program redirect.c

The program redirect.c does what the shell does for `ls -l > out.txt` (Figure 3.1). In the child it closes the descriptor 1, the standard output, then opens out.txt for writing, creating it if needed and emptying it (O_TRUNC). It prints the number that open returned to the standard error, so that the message is not caught by the redirection, and then runs ls. ls itself is not changed in any way: it writes to the descriptor 1, as always.

```
wancea@OS-Lab1-2:~/os-lab7$ cat redirect.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                                   /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Figure 3.1 – The file redirect.c

3.2 The result

The run printed `open gave descriptor 1` and `done: the listing is in out.txt`, but no listing (Figure 3.2). The listing was in the file: `head -3 out.txt` shows its first lines, total 76 and the entries out.txt and pipeline.c. The file out.txt appears in its own listing with the size 0, because ls read the folder while the file was still empty, before it had written anything into it.

```
wancea@OS-Lab1-2:~/os-lab7$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
wancea@OS-Lab1-2:~/os-lab7$ head -3 out.txt
total 76
-rw-r--r-- 1 wancea wancea     0 Oct 25 17:05 out.txt
-rw-rw-r-- 1 wancea wancea  1013 Oct 25 16:59 pipeline.c
```

Figure 3.2 – The listing of ls ends up in out.txt

3.3 Answers to the questions

1. open gave the descriptor 1. The kernel always gives a new descriptor the smallest number that is free in the process. The child had 0, 1 and 2 open on the terminal and had just closed 1, so 1 was the smallest free number.

2. ls writes its listing to the descriptor 1, and in the child 1 now refers to the file out.txt, not to the terminal. exec keeps the open descriptors of the process, so ls inherited this redirection without knowing about it. The message about the descriptor still appeared on the screen because it was written to the descriptor 2, which was not changed.

3. To redirect the errors, like `2> err.txt`, the child would close the descriptor 2 before open, so that open returns 2 and everything ls writes to the standard error goes into the file.

4 A PIPE BETWEEN LS AND WC

4.1 The program pipeline.c

The program pipeline.c builds the pipeline `ls /etc | wc -l` by hand. The parent creates one pipe, a buffer in the kernel with a reading end p[0] and a writing end p[1], and then two children. In the first child the descriptor 1 is closed and `dup(p[1])` copies the writing end into the smallest free number, which is 1; then both original ends are closed and the child becomes `ls /etc`. In the second child the descriptor 0 is replaced in the same way by the reading end, and the child becomes `wc -l`. The parent closes both ends, which it does not use, and waits for the two children [4], [5].

The program printed 237, the same number as the shell for `ls /etc | wc -l` (Figure 4.1). It is also the number of entries of /etc found in Q8 of the third laboratory work.

```
wancea@OS-Lab1-2:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
237
wancea@OS-Lab1-2:~/os-lab7$ ls /etc | wc -l
237
```

Figure 4.1 – pipeline.c and the shell give the same number

4.2 Breaking the pipe on purpose

The line `close(p[1]);` of the parent, line 33, was turned into a comment with sed (Figure 4.2). `grep -n 'close(p' pipeline.c` shows that the children still close both ends, and that only the parent keeps p[1] open. Compiled and run again, the program printed nothing and did not end, so it was stopped with Ctrl-C.

```
wancea@OS-Lab1-2:~/os-lab7$ sed -i 's|^    close(p\[1\]);$|    /* close(p[1]); */|'
 pipeline.c
wancea@OS-Lab1-2:~/os-lab7$ grep -n 'close(p' pipeline.c
15:        close(p[0]);
16:        close(p[1]);
25:        close(p[0]);
26:        close(p[1]);
32:    close(p[0]);                                /* the parent uses neither end
*/
33:    /* close(p[1]); */
wancea@OS-Lab1-2:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
^C
```

Figure 4.2 – The parent no longer closes p[1], and the program hangs

To see which process was stuck, the broken program was started again in the background and ps was run two seconds later (Figure 4.3). The parent ./pipeline (4443) and its child `wc -l` (4446) were both in the state S, waiting, while ls was no longer in the list: it had written its 237 lines into the pipe and ended, and the parent had already collected it with the first wait. The job was then stopped with `kill %1`.

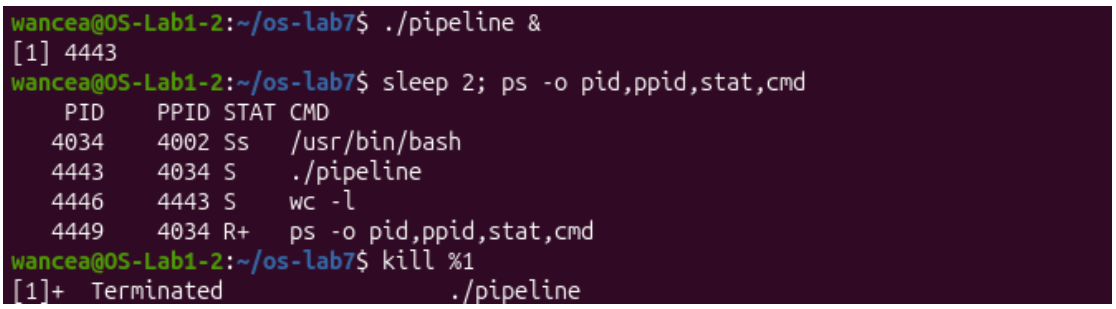


Figure 4.3 – After ls has ended, wc is still waiting

Afterwards ps showed only bash and ps itself, so no process of the test was left behind. The comment was removed with sed, and the restored program printed 237 again (Figure 4.4). The file pipeline.c in the repository is this final, working version.

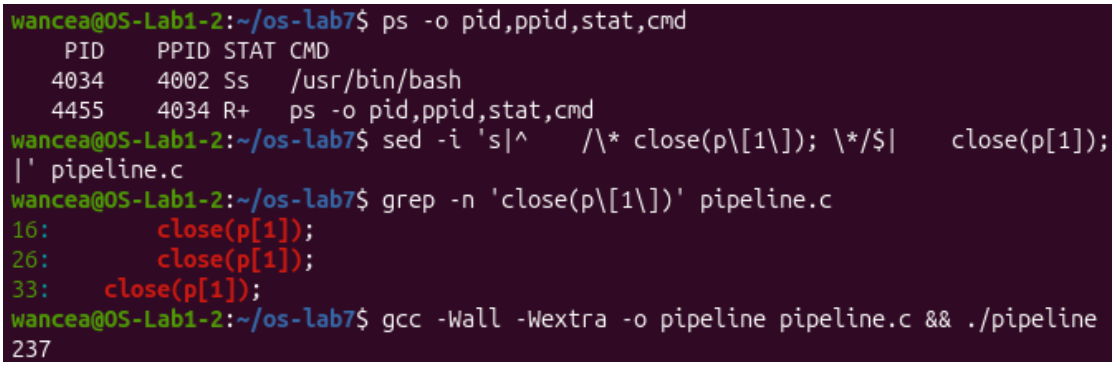


Figure 4.4 – No process is left, and the restored pipeline.c works again

4.3 Optional: three stages

The optional program pipeline3.c runs `ls /usr/bin | grep x | wc -l` with two pipes and three children: ls writes into the first pipe, grep reads the first pipe and writes into the second one, and wc reads the second pipe. Every child closes all four ends after its dup calls, and so does the parent. The program printed 227, the same number as the shell (Figure 4.5).

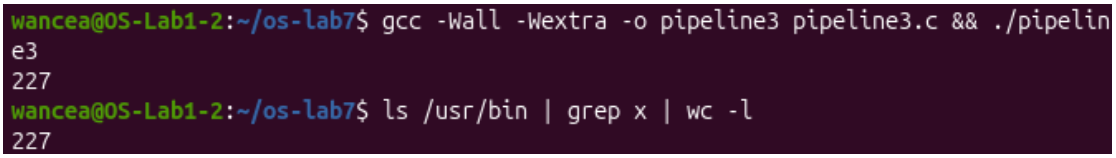


Figure 4.5 – pipeline3.c and the shell give the same number

4.4 Answers to the questions

1. Yes. Both ./pipeline and `ls /etc | wc -l` printed 237 (Figure 4.1).

2. After the change wc never ends. ls finishes normally and disappears, while `wc -l` stays in the state S (Figure 4.3).

3. wc reads until it gets the end of file, and a pipe gives the end of file only when every writing end of it is closed; the parent still holds p[1] open, and it never closes it because it is itself waiting for wc.

CONCLUSIONS

The work showed that the shell runs every command with three system calls. fork makes a copy of the process, exec replaces the program of the copy and keeps everything else, including the PID and the open descriptors, and wait collects the exit status of the child. The code after exec runs only when exec fails, which is why the child of run.c printed an error and the status 127 only for a program that does not exist.

The zombie showed what happens between the end of a child and the wait of its parent: the process is gone, but the kernel keeps its entry with the exit status until the parent asks for it. A parent that never calls wait would leave such entries behind for as long as it runs.

Redirection and pipes turned out to need no help from the programs. Because open and dup always take the smallest free descriptor, a child can close 0 or 1 and put a file or the end of a pipe in its place before exec, and ls or wc then use it without knowing. The broken pipeline showed the other side of this rule: the end of file comes only when every copy of the writing end is closed, so a single descriptor forgotten in the parent is enough to make wc wait forever.

REFERENCES

1. Three Small Programs: Processes, Redirection, a Pipe. Laboratory Work 7, Operating Systems course. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.

2. fork(2). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man2/fork.2.html

3. exec(3) and wait(2). Linux manual pages [online]. Available: https://man7.org/linux/man-pages/man3/exec.3.html, https://man7.org/linux/man-pages/man2/wait.2.html

4. pipe(2). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man2/pipe.2.html

5. dup(2). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man2/dup.2.html